

A Practical Course on Mathematical Modelling

Lecture 5 - Branch-and-Cut with Bonobo.jl

Prof. Viet Hung Nguyen

Clermont Auvergne University - Clermont-Ferrand - France

Reference knapsack

$$\max \quad 7x_1 + 10x_2 + 4x_3 + 4x_4 + 8x_5$$

$$3x_1 + 5x_2 + 12x_3 + 2x_4 + 6x_5 \leq 15, \quad x_i \in \{0, 1\}.$$

item i	1	2	3	4	5
profit p_i	7	10	4	4	8
weight w_i	3	5	12	2	6

The same instance is used first for dynamic programming, then for cutting planes, and finally for Branch-and-Cut.

Dynamic programming for the knapsack

Let $f_i(c)$ be the best profit using only items $1, \dots, i$ with capacity c .

$$f_0(c) = 0, \quad f_i(c) = \begin{cases} f_{i-1}(c) & \text{if } w_i > c, \\ \max\{f_{i-1}(c), f_{i-1}(c - w_i) + p_i\} & \text{if } w_i \leq c. \end{cases}$$

The two options for item i are: **skip it** or **take it**. The optimal value is $f_n(C)$.

On our instance ($C = 15$):

$$f_2(8) = \max\{f_1(8), f_1(8 - 5) + 10\} = \max\{7, 7 + 10\} = 17.$$

$$f_4(7) = \max\{f_3(7), f_3(7 - 2) + 4\} = \max\{10, 10 + 4\} = 14.$$

Complexity: $O(nC)$ time and memory, which is pseudo-polynomial in the input size.

Dynamic programming table for the reference knapsack

$f_i(c)$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
$i = 1 (3, 7)$	0	0	0	7	7	7	7	7	7	7	7	7	7	7	7	7
$i = 2 (5, 10)$	0	0	0	7	7	10	10	10	17	17	17	17	17	17	17	17
$i = 3 (12, 4)$	0	0	0	7	7	10	10	10	17	17	17	17	17	17	17	17
$i = 4 (2, 4)$	0	0	4	7	7	11	11	14	17	17	21	21	21	21	21	21
$i = 5 (6, 8)$	0	0	4	7	7	11	11	14	17	17	21	21	21	22	25	25

Row labels give (w_i, p_i) . Optimal value: $f_5(15) = 25$. **Backtracking** (shaded path):
 $f_5(15) = 25 \neq f_4(15) = 21 \Rightarrow$ take item 5, $c = 9$; $f_4(9) = f_3(9) = f_2(9) \Rightarrow$ skip items 4 and 3;
 $f_2(9) = 17 \neq f_1(9) = 7 \Rightarrow$ take item 2, $c = 4$; $f_1(4) = 7 \neq f_0(4) = 0 \Rightarrow$ take item 1.

$x = (1, 1, 0, 0, 1)$, weight $3 + 5 + 6 = 14 \leq 15$, profit 25. The LP bound is 27.67, so the integrality gap is 2.67.

Root LP relaxation

The binary conditions are relaxed:

$$0 \leq x_i \leq 1.$$

The LP relaxation gives

$$x_1 = x_2 = x_4 = 1, \quad x_3 = 0, \quad x_5 = \frac{5}{6}.$$

$$z_{LP} = 7 + 10 + 4 + 8 \cdot \frac{5}{6} = 27.67.$$

This point is not feasible for the binary model.

$$x_5 = \frac{5}{6} \notin \{0, 1\}.$$

The value 27.67 is nevertheless a valid upper bound.

Cutting-plane loop

A cutting-plane algorithm strengthens the relaxation without branching.

1. Solve the current LP relaxation.
2. If the solution is integer, stop.
3. Find a violated valid inequality.
4. Add it to the LP relaxation.
5. Re-solve.

A valid inequality removes fractional points but keeps all feasible integer points.

$$P_{\text{IP}} \subseteq P_{\text{LP}} \cap \{\text{new cuts}\}.$$

Generic integrality cuts: Chvatal–Gomory idea

Let

$$Ax \leq b, \quad x \in \mathbb{Z}_+^n.$$

For any nonnegative multiplier vector u such that $u^\top A$ is integer,

$$u^\top Ax \leq u^\top b$$

implies

$$u^\top Ax \leq \lfloor u^\top b \rfloor.$$

More generally, by combining original inequalities and variable bounds, one obtains valid inequalities based only on integrality.

linear combination + rounding = CG cut.

A CG cut for the knapsack

Start from the capacity constraint divided by 6:

$$\frac{1}{2}x_1 + \frac{5}{6}x_2 + 2x_3 + \frac{1}{3}x_4 + x_5 \leq \frac{15}{6}.$$

Add upper-bound inequalities:

$$\frac{1}{2}x_1 \leq \frac{1}{2}, \quad \frac{1}{6}x_2 \leq \frac{1}{6}, \quad \frac{2}{3}x_4 \leq \frac{2}{3}.$$

This gives

$$x_1 + x_2 + 2x_3 + x_4 + x_5 \leq \frac{23}{6}.$$

Since the left-hand side is integer,

$$x_1 + x_2 + 2x_3 + x_4 + x_5 \leq 3.$$

The CG cut separates the root solution

At the root LP solution,

$$x = (1, 1, 0, 1, 5/6).$$

For the CG cut,

$$x_1 + x_2 + 2x_3 + x_4 + x_5 = 1 + 1 + 0 + 1 + \frac{5}{6} = \frac{23}{6} > 3.$$

Therefore the cut removes the fractional root point.

After adding the cut, the strengthened relaxation has the integer optimum

$$x = (1, 1, 0, 0, 1), \quad z = 25.$$

Combinatorial cuts: cover inequalities

For a binary knapsack

$$\sum_i w_i x_i \leq b, \quad x_i \in \{0, 1\},$$

a set C is a cover if

$$\sum_{i \in C} w_i > b.$$

All items in C cannot be selected simultaneously.

Thus the cover inequality

$$\sum_{i \in C} x_i \leq |C| - 1$$

is valid.

A cover cut for the example

Take

$$C = \{1, 2, 4, 5\}.$$

Its total weight is

$$3 + 5 + 2 + 6 = 16 > 15.$$

Therefore

$$x_1 + x_2 + x_4 + x_5 \leq 3$$

is valid.

At the root LP solution,

$$x_1 + x_2 + x_4 + x_5 = 1 + 1 + 1 + \frac{5}{6} = 3.83 > 3.$$

The cover inequality is violated.

Effect of the cover cut

After adding

$$x_1 + x_2 + x_4 + x_5 \leq 3,$$

the relaxation bound becomes

$$\text{UB} = 25.33.$$

One optimal LP solution is

$$x_1 = x_2 = x_5 = 1, \quad x_3 = \frac{1}{12}, \quad x_4 = 0.$$

The relaxation is stronger, but still fractional.

$$25.33 > 25.$$

Extended cover inequality

Let C be a cover.

A standard extension is

$$E(C) = C \cup \left\{ j \notin C : w_j \geq \max_{i \in C} w_i \right\}.$$

Then

$$\sum_{i \in E(C)} x_i \leq |C| - 1$$

is valid.

The right-hand side remains $|C| - 1$, not $|E(C)| - 1$.

Extended cover for the example

For

$$C = \{1, 2, 4, 5\},$$

we have

$$\max_{i \in C} w_i = 6.$$

Since

$$w_3 = 12 \geq 6,$$

item 3 can be added to the extended cover.

$$x_1 + x_2 + x_3 + x_4 + x_5 \leq 3.$$

This cuts the fractional solution with $x_3 = 1/12$ and closes the root gap:

Two types of cuts

Cut type	Source	Character
CG cut	integrality and linear inequalities	generic
Cover cut	knapsack structure	combinatorial
Extended cover	strengthened cover	combinatorial

A CG cut may be derived without interpreting the constraint as a knapsack.

A cover inequality exploits the combinatorial meaning of the capacity constraint.

Larger knapsack instance

We now use a larger instance to illustrate cuts generated away from the root.

$$\max 13x_1 + 10x_2 + 7x_3 + 6x_4 + 12x_5 + 11x_6 + 9x_7 + 8x_8$$

$$6x_1 + 5x_2 + 4x_3 + 4x_4 + 7x_5 + 6x_6 + 5x_7 + 5x_8 \leq 18.$$

$$x_i \in \{0, 1\}.$$

The root LP solution is

$$x = (1, 1, 0, 0, 0, 1, 0.2, 0), \quad \text{UB} = 35.8.$$

A non-root node

Branch on the fractional variable x_7 .

Consider the child node

$$x_7 = 1.$$

The LP relaxation at this node gives

$$x = (1, 1, 0, 0, 0, 1/3, 1, 0), \quad \text{UB} = 35.67.$$

This node is fractional because

$$x_6 = \frac{1}{3}.$$

The remaining capacity after fixing $x_7 = 1$ is

$$18 - 5 = 13.$$

A local CG cut at the node $x_7 = 1$

At the node $x_7 = 1$, the residual capacity is

$$6x_1 + 5x_2 + 4x_3 + 4x_4 + 7x_5 + 6x_6 + 5x_8 \leq 13.$$

Multiply by $1/5$:

$$\frac{6}{5}x_1 + x_2 + \frac{4}{5}x_3 + \frac{4}{5}x_4 + \frac{7}{5}x_5 + \frac{6}{5}x_6 + x_8 \leq \frac{13}{5}.$$

Rounding down the integer left-hand side gives the local CG cut

$$x_1 + x_2 + x_5 + x_6 + x_8 \leq 2.$$

The local CG cut is violated

At the node solution

$$x = (1, 1, 0, 0, 0, 1/3, 1, 0),$$

the left-hand side of the local CG cut is

$$x_1 + x_2 + x_5 + x_6 + x_8 = 1 + 1 + 0 + \frac{1}{3} + 0 = 2.33.$$

$$2.33 > 2.$$

The cut is generated at a non-root node.

It is local because it uses the restriction $x_7 = 1$.

A local cover cut at the same node

At the node $x_7 = 1$, the residual capacity is 13.

Take

$$C = \{1, 2, 6\}.$$

Its residual weight is

$$6 + 5 + 6 = 17 > 13.$$

Thus, inside this node,

$$x_1 + x_2 + x_6 \leq 2$$

is valid.

At the node LP solution,

$$x_1 + x_2 + x_6 = 1 + 1 + \frac{1}{2} = 2.33 > 2.$$

A local extended cover cut

For the local cover

$$C = \{1, 2, 6\},$$

we have

$$\max_{i \in C} w_i = 6.$$

Item 5 has weight 7, hence it can be added.

$$x_1 + x_2 + x_5 + x_6 \leq 2.$$

This is valid only in the subtree $x_7 = 1$.

It is not a global inequality for the original knapsack.

Bonobo Branch-and-Cut: main idea

Bonobo manages the Branch-and-Bound tree.

The cut loop is placed inside the node evaluation.

1. apply node restrictions,
2. solve relaxation,
3. separate violated cuts,
4. add cuts and re-solve,
5. return the final node bound.

For local cuts, the node must store the cuts valid in its subtree.

Global cuts versus local cuts

Global cuts are valid for the original problem.

$$\alpha^T x \leq \beta \quad \forall x \in X.$$

They can be kept permanently.

Local cuts are valid only after some branching decisions.

$$\alpha^T x \leq \beta \quad \forall x \in X \cap N.$$

They must be stored in the node and inherited by its descendants.

For pedagogical implementations, rebuilding the LP model at each node is often clearer than adding and deleting local constraints in a shared model.

Node type with local cuts

```
struct LinearCut
  coeff::Vector{Float64}
  rhs::Float64
  name::String
end

mutable struct CutNode <: BB.AbstractNode
  std::BB.BnBNodeInfo
  lbs::Vector{Float64}
  ubs::Vector{Float64}
  local_cuts::Vector{LinearCut}
  lp_objective::Float64
  lp_solution::Vector{Float64}
  integral::Bool
  cuts_added_at_node::Int
end
```

The field `local_cuts` contains cuts inherited from the ancestors.

Building the node relaxation

```
function build_node_model(node::CutNode)
    model = Model(HiGHS.Optimizer)
    set_silent(model)

    @variable(model, node.lbs[i] <= x[i=1:n] <= node.ubs[i])

    @constraint(model,
        sum(weight[i] * x[i] for i in 1:n) <= capacity
    )

    for cut in node.local_cuts
        @constraint(model,
            sum(cut.coeff[i] * x[i] for i in 1:n) <= cut.rhs
        )
    end

    @objective(model, Max,
        sum(profit[i] * x[i] for i in 1:n)
    )

    return model, x
end
```

Residual capacity at a node

For a knapsack node, fixed variables can be detected from the bounds.

```
function residual_capacity(node)
    fixed_one = [
        i for i in 1:n
        if node.lbs[i] == 1 && node.ubs[i] == 1
    ]

    return capacity - sum(weight[i] for i in fixed_one)
end

function free_indices(node)
    return [
        i for i in 1:n
        if !(node.lbs[i] == node.ubs[i])
    ]
end
```

Local cuts are generated on the residual subproblem.

CG separator for a knapsack node

For a multiplier $\theta > 0$, the residual CG cut is

$$\sum_{i \in F} \lfloor \theta w_i \rfloor x_i \leq \lfloor \theta b_N \rfloor.$$

```
function separate_cg_cuts(node, xval; theta_values = [1/2, 1/3, 1/4, 1/5, 1/6])
    bN = residual_capacity(node)
    F = free_indices(node)
    cuts = LinearCut[]

    for theta in theta_values
        coeff = zeros(n)
        for i in F
            coeff[i] = floor(theta * weight[i])
        end
        rhs = floor(theta * bN)

        if sum(coeff[i] * xval[i] for i in 1:n) > rhs + 1e-6
            push!(cuts, LinearCut(coeff, rhs, "CG theta=$(theta)"))
        end
    end

    return cuts
end
```

CG Branch-and-Cut node evaluation: solve

```
function BB.evaluate_node!(tree, node::CutNode)
    node.cuts_added_at_node = 0

    while true
        model, x = build_node_model(node)
        optimize!(model)

        if termination_status(model) != MOI.OPTIMAL
            return NaN, NaN
        end

        node.lp_objective = objective_value(model)
        node.lp_solution = value.(x)

        # separation on the next slide
    end
end
```

The node model is rebuilt after each new local cut.

CG Branch-and-Cut node evaluation: separate

```
new_cuts = separate_cg_cuts(node, node.lp_solution)

if isempty(new_cuts)
    break
end

append!(node.local_cuts, new_cuts)
node.cuts_added_at_node += length(new_cuts)
```

After appending the cuts, the relaxation is rebuilt and re-solved.

At the node $x_7 = 1$, this separator can generate

$$x_1 + x_2 + x_5 + x_6 + x_8 \leq 2.$$

CG Branch-and-Cut node evaluation: return

```
node.integral = all(  
    BB.is_approx_feasible.(tree, node.lp_solution)  
)  
  
if node.integral  
    return node.lp_objective, node.lp_objective  
else  
    return node.lp_objective, NaN  
end
```

The returned bound is the value after all separated CG cuts have been added.

Children inherit local cuts

When a node is branched, its children inherit its local cuts.

```
push!(children, (  
    lbs = child_lbs,  
    ubs = child_ubs,  
    local_cuts = copy(node.local_cuts),  
    lp_objective = NaN,  
    lp_solution = fill(NaN, n),  
    integral = false,  
    cuts_added_at_node = 0,  
))
```

A cut valid in a node remains valid in every descendant of that node.

Cover-separation logic

At a node, fixed-one variables reduce the residual capacity.

$$b_N = b - \sum_{i: x_i=1 \text{ fixed}} w_i.$$

A local cover is a set C of free variables satisfying

$$\sum_{i \in C} w_i > b_N.$$

The corresponding local cut is

$$\sum_{i \in C} x_i \leq |C| - 1.$$

Cover cut representation

```
struct CoverCut
    support::Vector{Int}
    rhs::Int
    name::String
end

function to_linear_cut(cut::CoverCut)
    coeff = zeros(n)
    for i in cut.support
        coeff[i] = 1.0
    end
    return LinearCut(coeff, cut.rhs, cut.name)
end
```

A simple cover separator

A minimal separator can enumerate small covers.

```
function separate_cover_cuts(node, xval; max_size = 4)
    bN = residual_capacity(node)
    F = free_indices(node)
    cuts = CoverCut[]

    for k in 2:max_size
        for C in combinations(F, k)
            if sum(weight[i] for i in C) > bN
                lhs = sum(xval[i] for i in C)
                if lhs > k - 1 + 1e-6
                    push!(cuts, CoverCut(collect(C), k - 1, "cover"))
                end
            end
        end
    end

    return cuts
end
```

For larger instances, exact separation is usually modeled as an auxiliary knapsack problem.

Extending a cover

```
function extend_cover(node, C)
  F = free_indices(node)
  threshold = maximum(weight[i] for i in C)

  E = copy(C)

  for j in F
    if !(j in C) && weight[j] >= threshold
      push!(E, j)
    end
  end

  return sort(unique(E))
end
```

The right-hand side stays $|C| - 1$, not $|E| - 1$.

Cover and extended-cover separator

```
function separate_extended_cover_cuts(node, xval)
    base_cuts = separate_cover_cuts(node, xval)
    cuts = LinearCut[]

    for cut in base_cuts
        C = cut.support
        E = extend_cover(node, C)

        rhs = length(C) - 1
        ext_cut = CoverCut(E, rhs, "extended cover")

        if sum(xval[i] for i in E) > rhs + 1e-6
            push!(cuts, to_linear_cut(ext_cut))
        else
            push!(cuts, to_linear_cut(cut))
        end
    end

    return cuts
end
```

Node cut on the larger example

At the node $x_7 = 1$, the residual capacity is 13.

The set

$$C = \{1, 2, 6\}$$

is a local cover:

$$6 + 5 + 6 = 17 > 13.$$

It gives

$$x_1 + x_2 + x_6 \leq 2.$$

Since $w_5 = 7 \geq 6$, the extended cover is

$$x_1 + x_2 + x_5 + x_6 \leq 2.$$

Violation at the non-root node

The LP solution at the node $x_7 = 1$ is

$$x = (1, 1, 0, 0, 0, 1/3, 1, 0).$$

For the cover inequality,

$$x_1 + x_2 + x_6 = 2.33 > 2.$$

For the extended cover inequality,

$$x_1 + x_2 + x_5 + x_6 = 2.33 > 2.$$

A local cover cut is generated inside the B&B tree, not only at the root.

Cover Branch-and-Cut node evaluation

The code structure is the same as for CG cuts.

```
function BB.evaluate_node!(tree, node::CutNode)
    node.cuts_added_at_node = 0

    while true
        model, x = build_node_model(node)
        optimize!(model)

        if termination_status(model) != MOI.OPTIMAL
            return NaN, NaN
        end

        node.lp_objective = objective_value(model)
        node.lp_solution = value.(x)

        new_cuts = separate_extended_cover_cuts(node, node.lp_solution)

        isempty(new_cuts) && break

        append!(node.local_cuts, new_cuts)
        node.cuts_added_at_node += length(new_cuts)
    end

    # integrality test and return as before
end
```

Where the two algorithms differ

Component	CG Branch-and-Cut	Cover Branch-and-Cut
cut source	rounding	knapsack structure
separator	multiplier scan	cover search
node data	local linear cuts	local cover cuts
cut validity	local or global	local or global
strength	generic	problem-specific

The surrounding Bonobo code is essentially the same.

Only the separator changes.

Local cuts in a shared-model implementation

If the same JuMP model is reused for all nodes, global cuts are easy:

add once and keep permanently.

Local cuts are more delicate:

valid only in a subtree.

Possible implementations:

- rebuild the node LP relaxation from scratch;
- add local cuts temporarily and delete them after node evaluation;
- maintain cut activation constraints;
- store local cuts in each node and pass them to descendants.

Rebuilding the relaxation is the clearest method for a teaching notebook.

Generic Branch-and-Cut template in Bonobo

1. Select a node N .
2. Build the relaxation of N .
3. Add inherited local cuts.
4. Solve the relaxation.
5. Separate violated cuts.
6. If cuts are found, store them in N and repeat.
7. Return the final node bound.
8. Let Bonobo fathom, update incumbent, or branch.
9. When branching, copy local cuts to children.

Logging useful information

For a Branch-and-Cut notebook, the callback should record:

- node identifier and parent identifier;
- branching restriction;
- final LP bound after cuts;
- number of cuts added at the node;
- incumbent value after processing the node;
- fathoming reason.

The tree visualization can then distinguish:

pure branching nodes

cut-generation nodes

fathomed nodes.

Summary

Cutting planes strengthen relaxations.

Branch-and-Cut places the cutting-plane loop inside Branch-and-Bound nodes.

For the small knapsack:

CG cuts and extended covers close the root gap.

For the larger knapsack:

local cuts can be generated after branching.

In Bonobo:

tree management is provided by Bonobo; cut separation is implemented by the user.

When the complete formulation is too large

Many integer programs have a compact set of variables and an exponential family of constraints.

A standard formulation has the form

$$\min\{c^T x : Ax \leq b, Hx \leq h, x \in \mathbb{Z}^n\},$$

where $Ax \leq b$ is small, but $Hx \leq h$ contains exponentially many valid inequalities.

Instead of writing all constraints at once, one starts from the small relaxation

$$Ax \leq b,$$

and generates missing constraints only when they are needed.

This is the principle of lazy-constraint generation.

TSP: a compact relaxation is not enough

For the symmetric traveling salesman problem, let

$$x_e = 1 \iff e \text{ belongs to the tour.}$$

The degree equations are compact:

$$\sum_{e \in \delta(v)} x_e = 2, \quad v \in V.$$

But they only force every vertex to have degree two.

They can produce several disjoint cycles instead of one Hamiltonian cycle.

To forbid subtours, one needs the subtour elimination constraints

$$\sum_{e \in E(S)} x_e \leq |S| - 1, \quad \emptyset \neq S \subsetneq V.$$

There are exponentially many such subsets S .

Lazy cuts: constraints generated at integer candidates

A lazy cut is a valid constraint that is part of the intended integer formulation, but is not inserted initially.

It is separated when the solver finds a candidate integer solution.

candidate integer solution $\implies$ check missing constraints

violated constraint found $\implies$ add it lazily and reject the candidate.

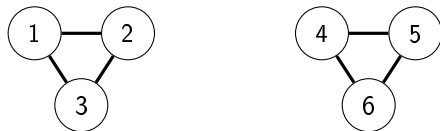
For TSP, the candidate solution may satisfy all degree equations but contain two or more subtours. Each subtour gives a violated subtour elimination constraint.

If no violated lazy constraint is found, the candidate is feasible for the full exponential formulation.

Example: two subtours in a TSP candidate

Suppose the degree equations return two disjoint cycles:

$$1 - 2 - 3 - 1, \quad 4 - 5 - 6 - 4.$$



For $S = \{1, 2, 3\}$, the subtour cut is

$$x_{12} + x_{13} + x_{23} \leq 2.$$

For $S = \{4, 5, 6\}$, the subtour cut is

$$x_{45} + x_{46} + x_{56} \leq 2.$$

These cuts remove the incumbent candidate, but keep all Hamiltonian tours.

Lazy separation loop for the TSP

1. Solve the MIP with degree equations and $x_e \in \{0, 1\}$.
2. When an integer candidate $\bar{x}$ appears, build the graph $G(\bar{x})$.
3. Find the connected components of $G(\bar{x})$.
4. If there is one component, $\bar{x}$ is a Hamiltonian tour.
5. If there are several components $S_1, \dots, S_k$, add

$$\sum_{e \in E(S_r)} x_e \leq |S_r| - 1, \quad r = 1, \dots, k.$$

6. Continue the branch-and-bound search.

The solver never needs to enumerate all subsets $S \subset V$ in advance.

Lazy cuts versus ordinary cutting planes

	Ordinary cutting plane	Lazy cut
Main role	Strengthen the LP relaxation	Complete a formulation with missing constraints
Separated at	Usually fractional LP solutions	Usually integer candidate solutions
Can remove an incumbent candidate?	No, if it is feasible for the full model	Yes, if it violates an omitted constraint
Typical examples	CG cuts, MIR cuts, cover cuts, clique cuts	TSP subtour cuts, connectivity cuts, capacity cuts in vehicle routing, cutset constraints in network design

In practice, Branch-and-Cut may use both mechanisms:

user cuts for bounds + lazy cuts for correctness.

Other problems with useful lazy cuts

Lazy cuts are natural when feasibility has a global combinatorial condition.

Problem	Typical lazy constraints
TSP	subtour elimination constraints
Vehicle routing	capacity cuts and route-elimination cuts
Steiner tree	connectivity cuts separating terminals from the root
Survivable network design	cutset inequalities enforcing required connectivity
Benders decomposition	feasibility or optimality cuts generated from a sub-problem

The common scheme is to solve a smaller master model, inspect the candidate solution, and add only constraints that are actually violated.

MTZ formulation for the directed TSP

For the asymmetric TSP, let $x_{ij} = 1$ when the tour uses arc $i \rightarrow j$. The degree equations choose one incoming and one outgoing arc at each city:

$$\sum_{j \neq i} x_{ij} = 1, \quad \sum_{j \neq i} x_{ji} = 1 \quad (i = 1, \dots, n), \quad x_{ij} \in \{0, 1\}.$$

These equations can allow several disjoint directed cycles. MTZ adds a visit order u_i for every city other than the depot 1:

$$1 \leq u_i \leq n - 1 \quad (i = 2, \dots, n),$$
$$u_i - u_j + (n - 1)x_{ij} \leq n - 2 \quad (i \neq j, i, j = 2, \dots, n).$$

When $x_{ij} = 1$, the inequality forces $u_j \geq u_i + 1$. A directed subtour among cities $2, \dots, n$ would require the visit order to increase around a cycle, which is impossible. The u_i variables may be continuous.

Desrochers–Laporte lifted MTZ constraints

Desrochers and Laporte strengthen each MTZ inequality by including the reverse arc variable:

$$u_i - u_j + (n - 1)x_{ij} + (n - 3)x_{ji} \leq n - 2 \quad (i \neq j, i, j = 2, \dots, n).$$

The coefficient $n - 3$ comes from lifting x_{ji} . If $x_{ji} = 1$, then arc $j \rightarrow i$ is used, so i immediately follows j and $u_i - u_j = 1$. The original inequality has slack $n - 3$ in this case, so that is the largest valid coefficient for x_{ji} .

For fractional x , the added nonnegative term tightens the LP relaxation. The formulation still has only $O(n^2)$ constraints, although its relaxation is generally weaker than the full subtour-elimination formulation.

Richard Lusby and Thomas Stidsen, *Mathematical Programming with Julia*, Section 5.2, “Solution algorithm: Branch & Bound”.

Bonobo.jl documentation:

<https://wikunia.github.io/Bonobo.jl/>

JuMP documentation:

<https://jump.dev/JuMP.jl/stable/>

For background on cover inequalities and Chvatal–Gomory cuts, see standard textbooks on integer programming and polyhedral combinatorics.